

Pertemuan 10 : AVL Tree

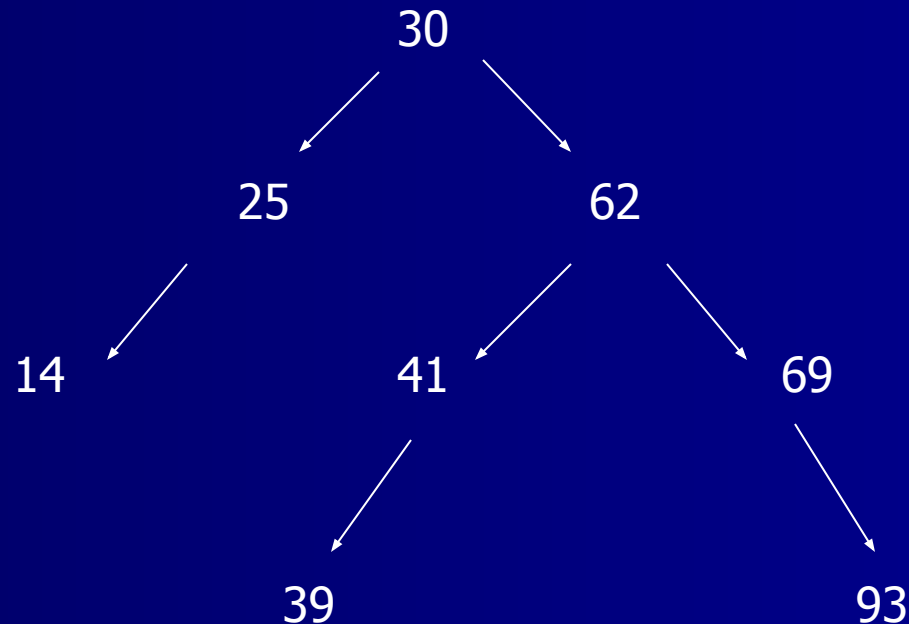
Kuliah Algoritma dan Struktur Data
Sekolah Vokasi IPB

Review Binary Search Tree

- Struktur Binary Tree banyak digunakan untuk membantu memecahkan persoalan suatu algoritma, contohnya untuk memecahkan masalah organisasi file direct.
- Penggunaan Binary Tree sebagai suatu struktur file yang disebut *Binary Search Tree (BST)*.
- Aturan Insert ke dalam File BST :
 - < Node □ Masuk Ke Kiri
 - > Node □ Masuk Ke Kanan

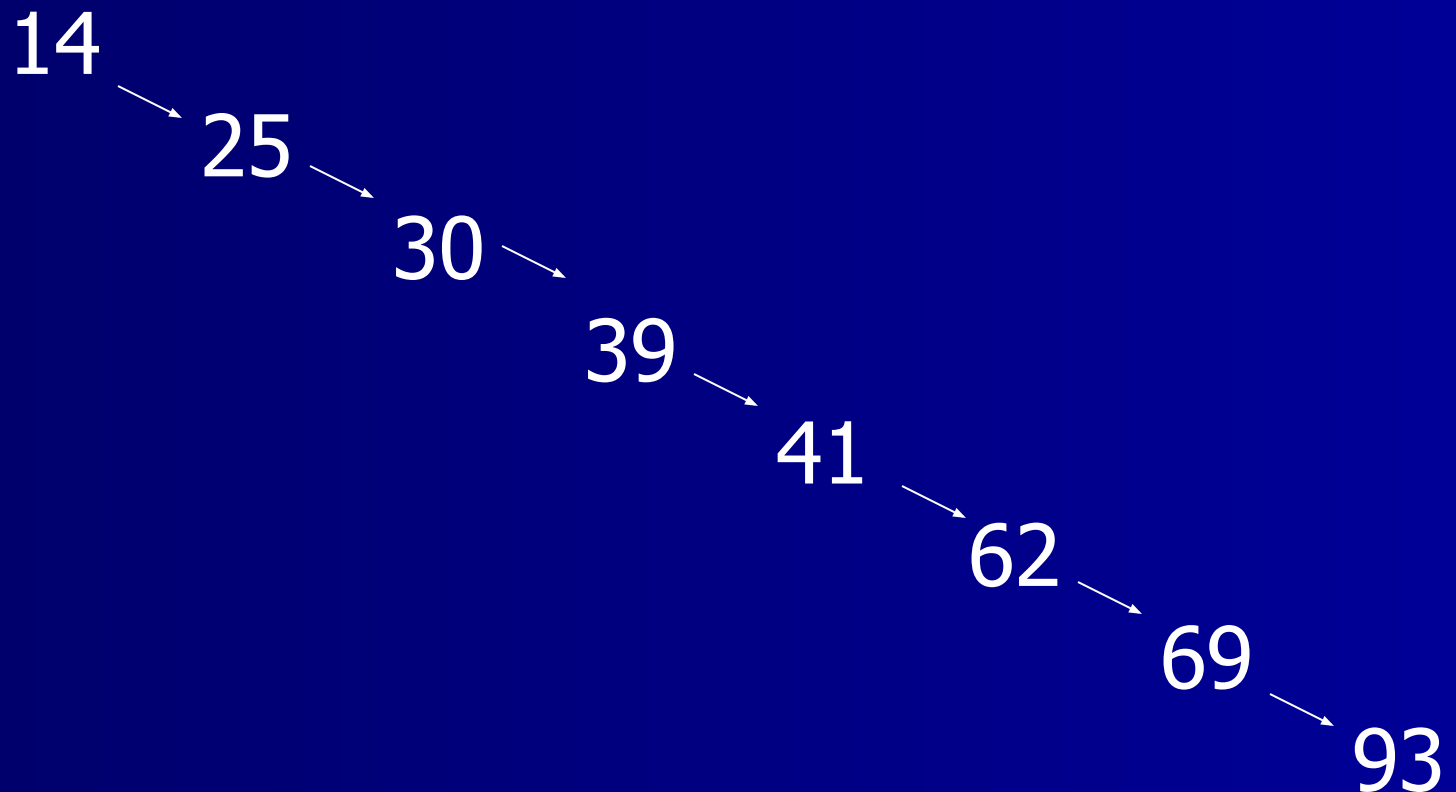
Contoh Insert ke Binary Search Tree

Key : 30, 62, 69, 41, 25, 39, 14, 93



Ketidakefisienan Binary Search Tree

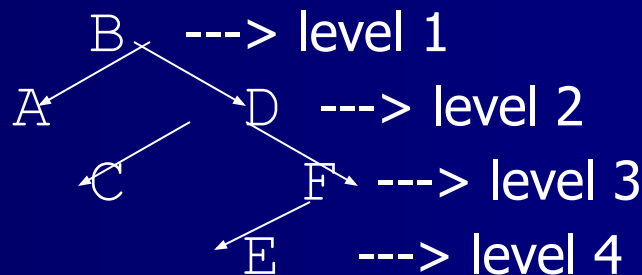
Key : 14, 25, 30, 39, 41, 62, 69, 93



AVL TREE (Adelson, Velskii, Landis)

BST Yang "Seimbang"

Balance Factor



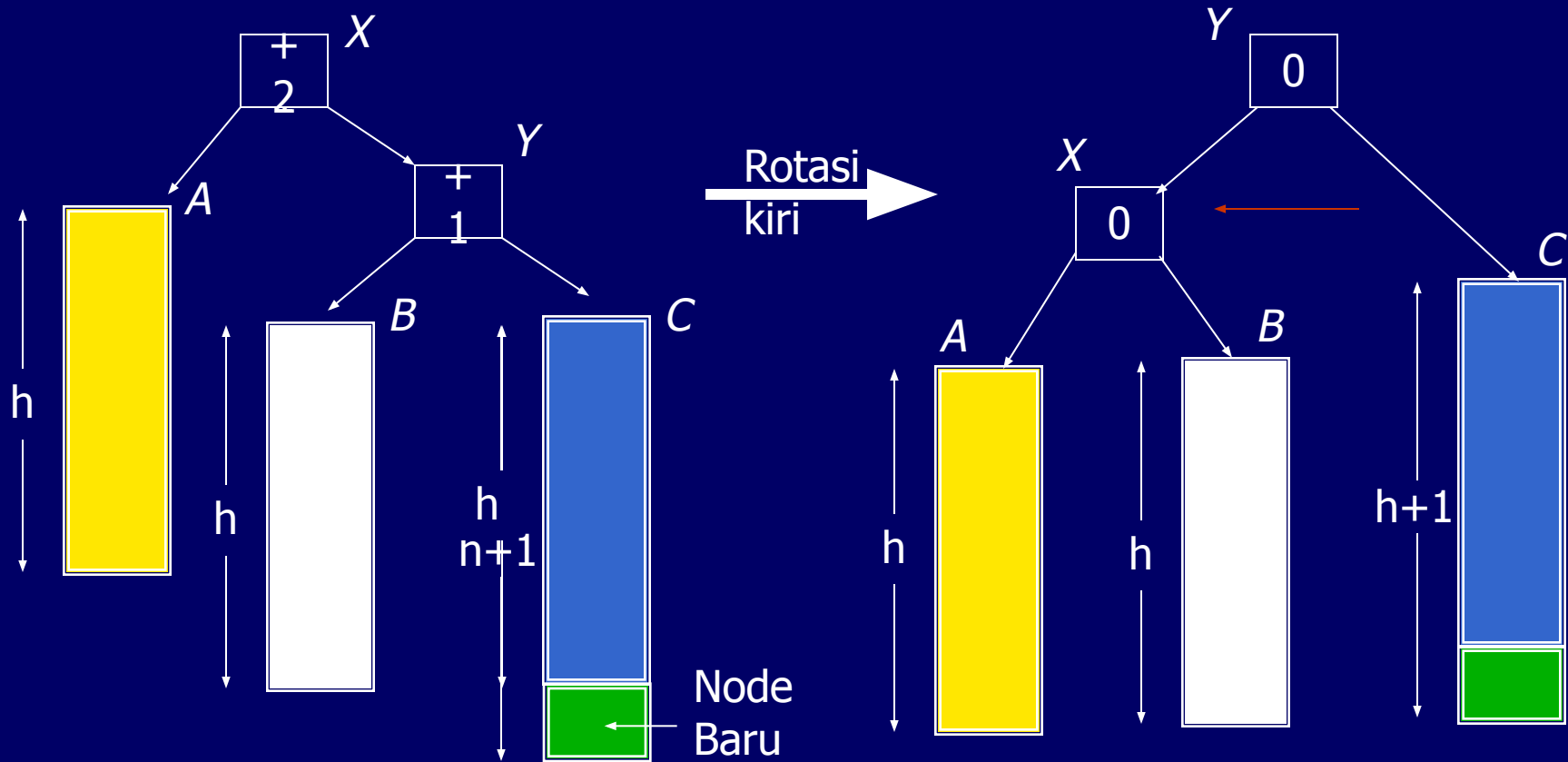
- Subtree : Tree yang terbentuk disebelah kanan atau kiri dari suatu node. (Catatan : Node tsb disebut root subtree)
- Tinggi subtree : Level node terbawah dari subtree dikurang level root subtree tsb
- Balance Factor : Tinggi subtree kanan dikurang tinggi dari subtree kiri dari suatu node

AVL Tree

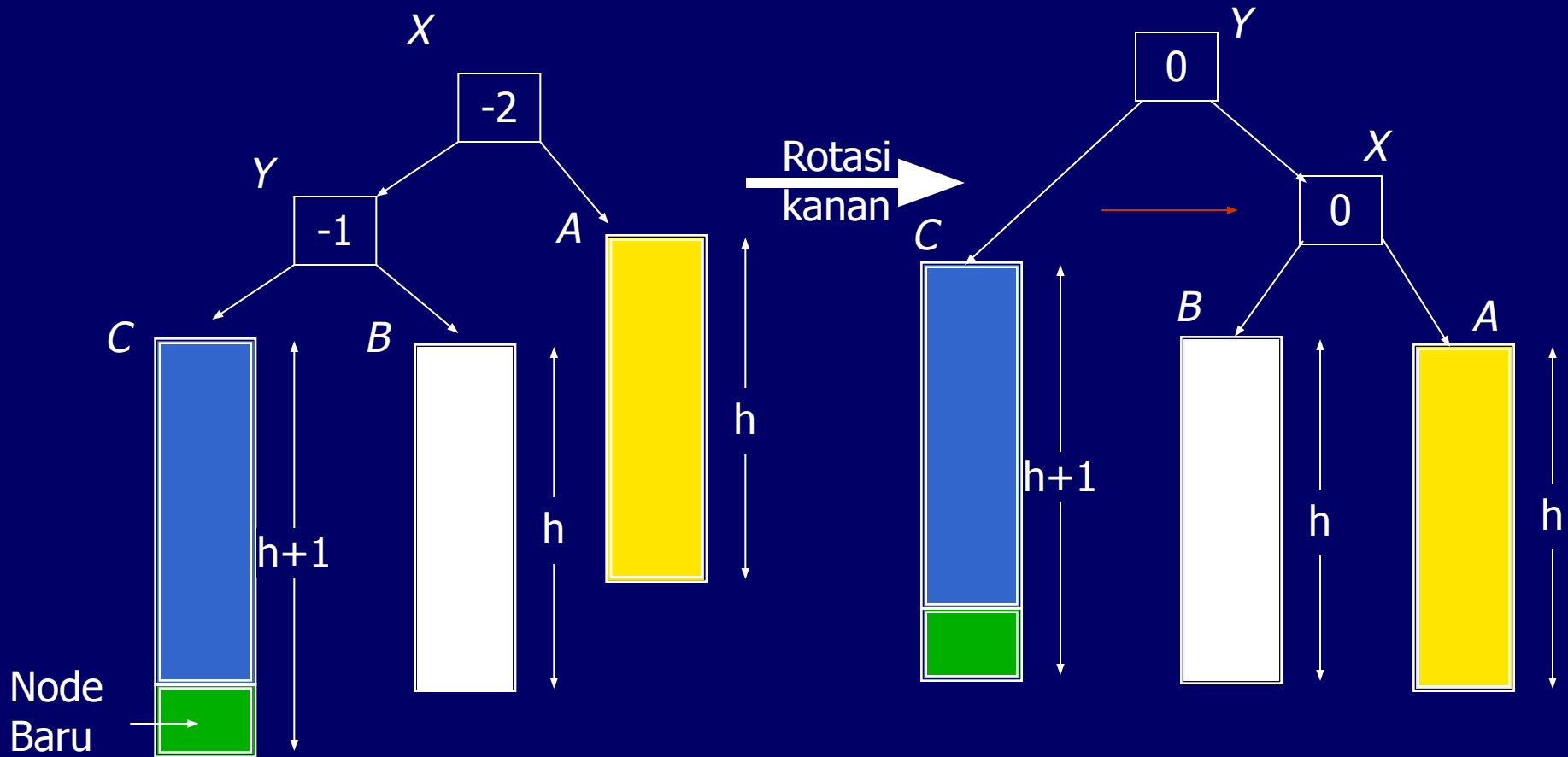
- Binary Search Tree yang memiliki aturan setiap node selalu memiliki balance factor 0, +1 atau -1
- Maka : setiap kali key baru diinsert ke dalam file, setiap node dilihat balance factornya
- Apabila ada node yang balance factornya tidak 0, +1 atau -1 (tidakimbang) maka struktur file dilakukan transformasi :
pemutaran dan atau **penggeseran**
- Kemungkinan tidakimbang hanya 2 dan diselesaikan dengan solusi : Rotasi Tunggal dan Rotasi Ganda

Ilustrasi 1

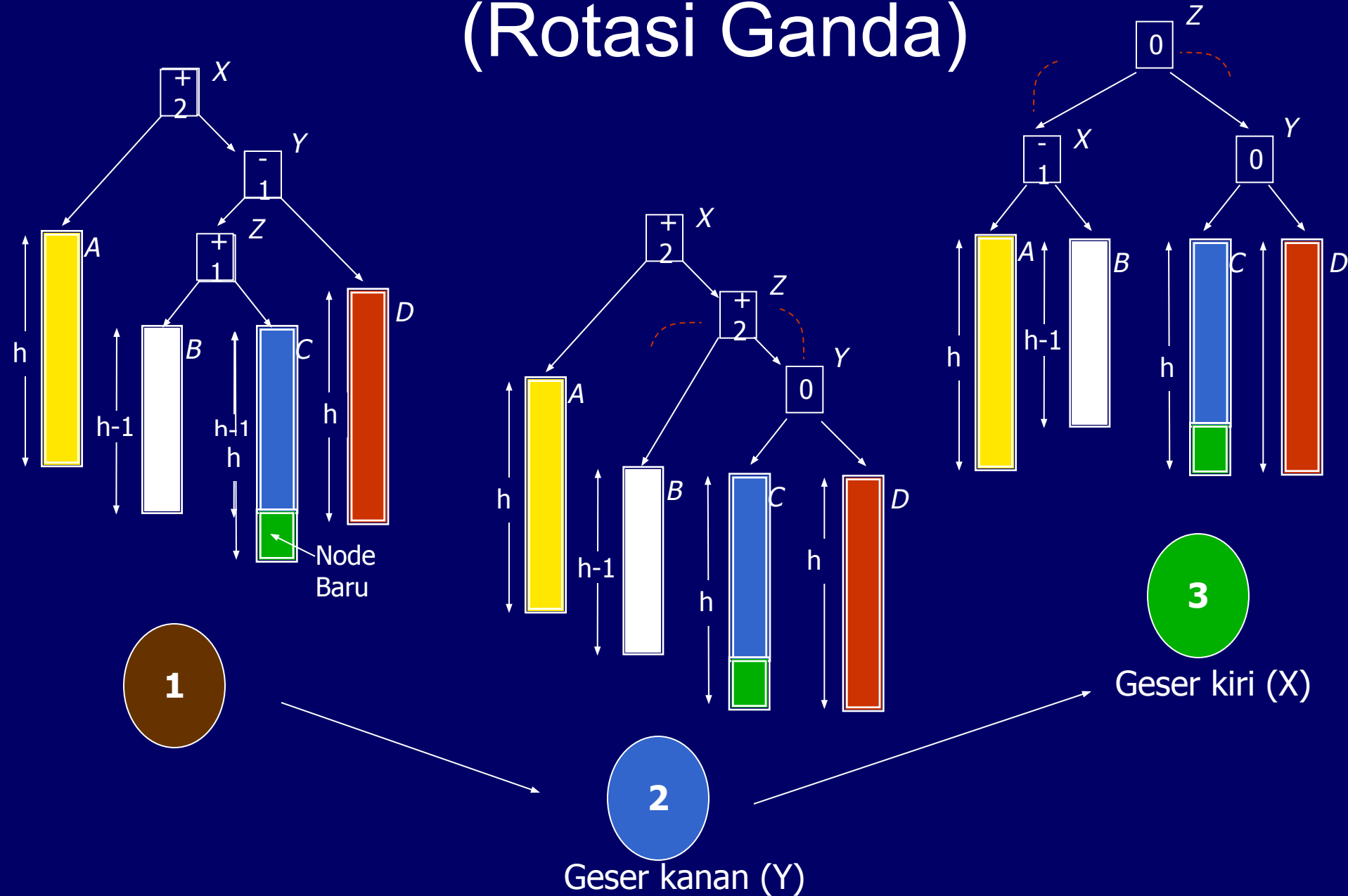
Kemungkinan Transformasi 1 (Rotasi Tunggal)



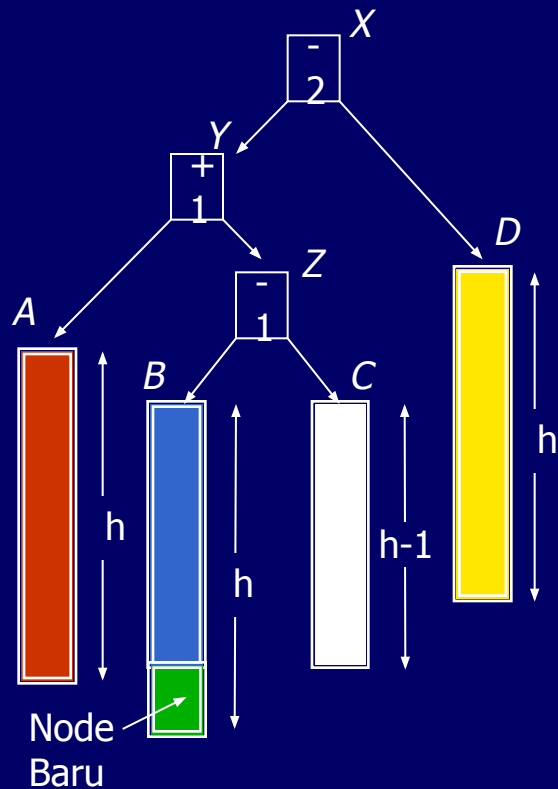
Kemungkinan Transformasi 1 (Rotasi Tunggal)



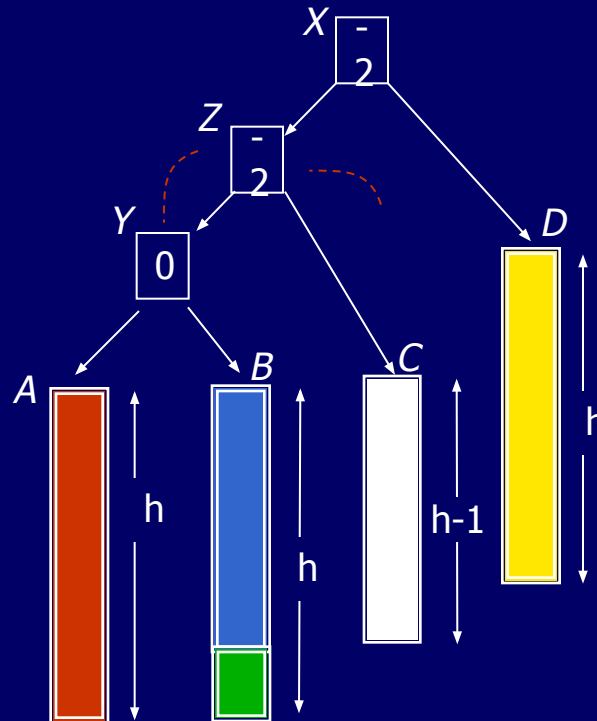
Kemungkinan Transformasi 2 (Rotasi Ganda)



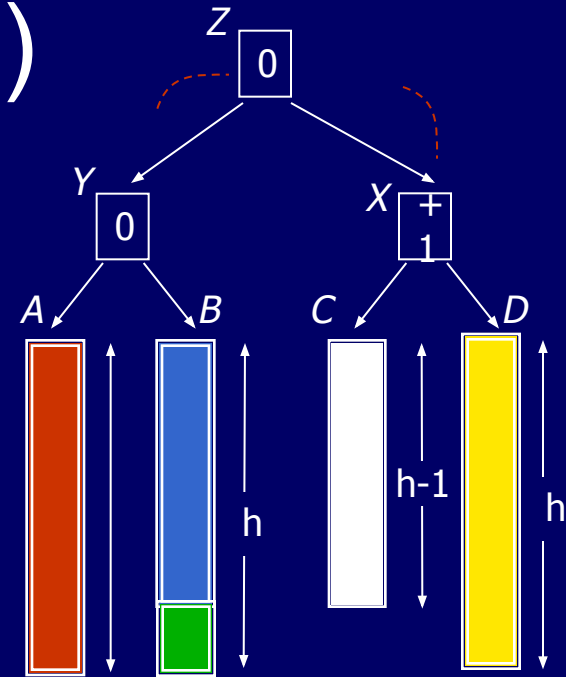
Kemungkinan Transformasi 2 (Rotasi Ganda)



1



2
Geser kiri (Y)

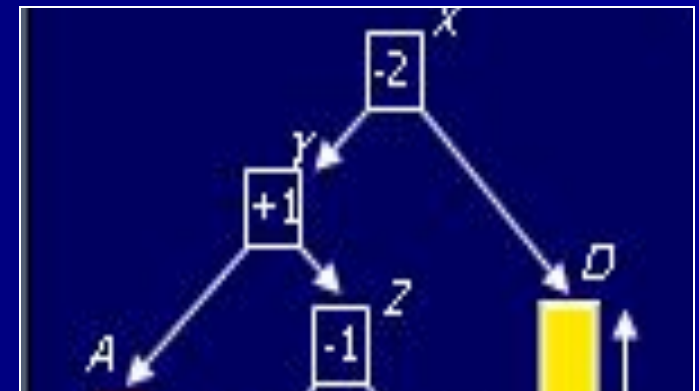
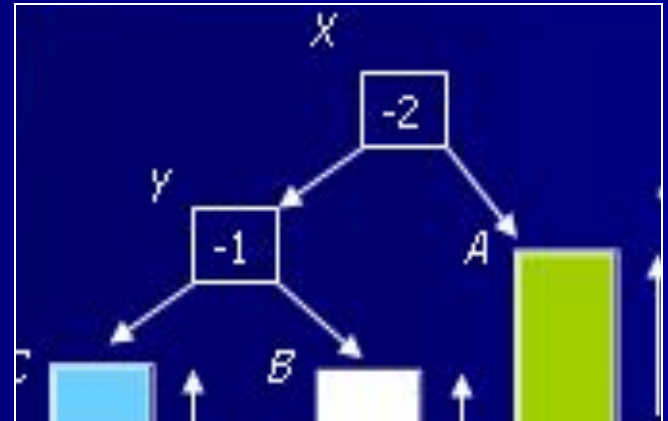


3

Geser kanan (X)

Transformasi Mana ?

- Bila 2 node teratas tandanya sama maka gunakan Transformasi 1 (rotasi tunggal)
- Bila 2 node teratas tandanya beda maka gunakan Transformasi 2 (rotasi ganda)

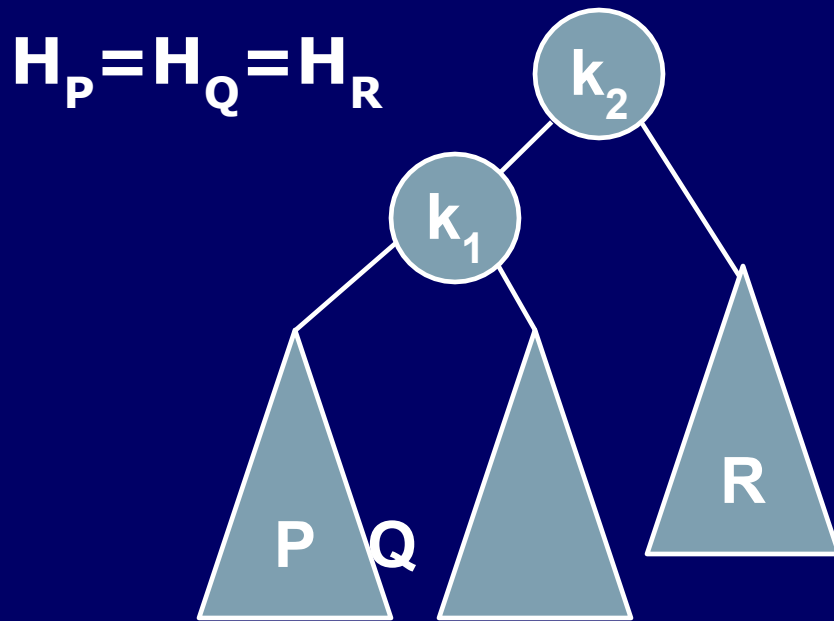


Ilustrasi 2

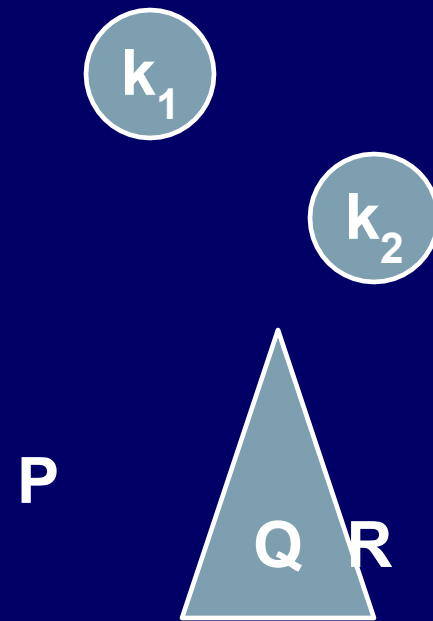
Insertion pada AVL Tree

- Untuk menjamin kondisi *balance* pada AVL tree, setelah penambahan sebuah node. jalur dari node baru tersebut hingga root di simpan dan di periksa kondisi *balance* pada tiap node-nya.
- Jika setelah penambahan, kondisi *balance* tidak terpenuhi pada node tertentu, maka lakukan salah satu rotasi berikut:
 - *Single rotation*
 - *Double rotation*

Kondisi tidak *balance*



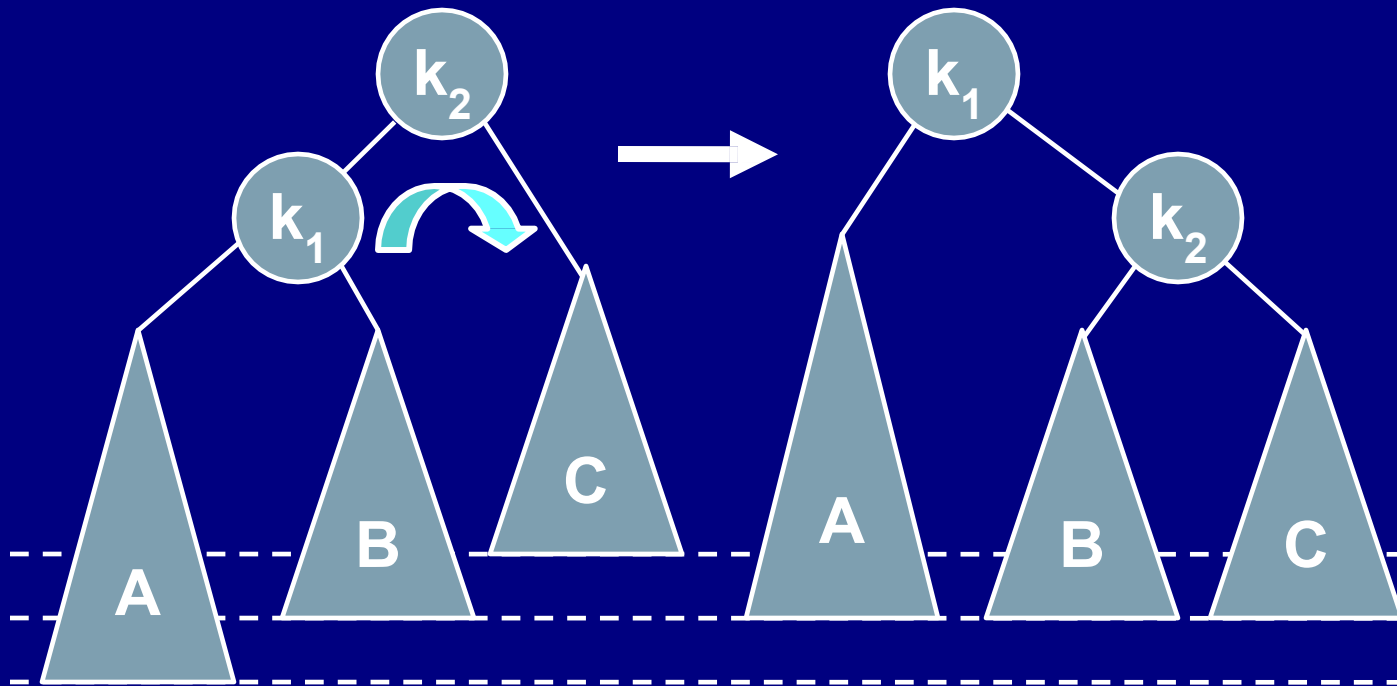
- Sebuah penambahan pada subtree:
 - P (outside) - case 1
 - Q (inside) - case 2



- Sebuah penambahan pada subtree:
 - Q (inside) - case 3
 - R (outside) - case 4

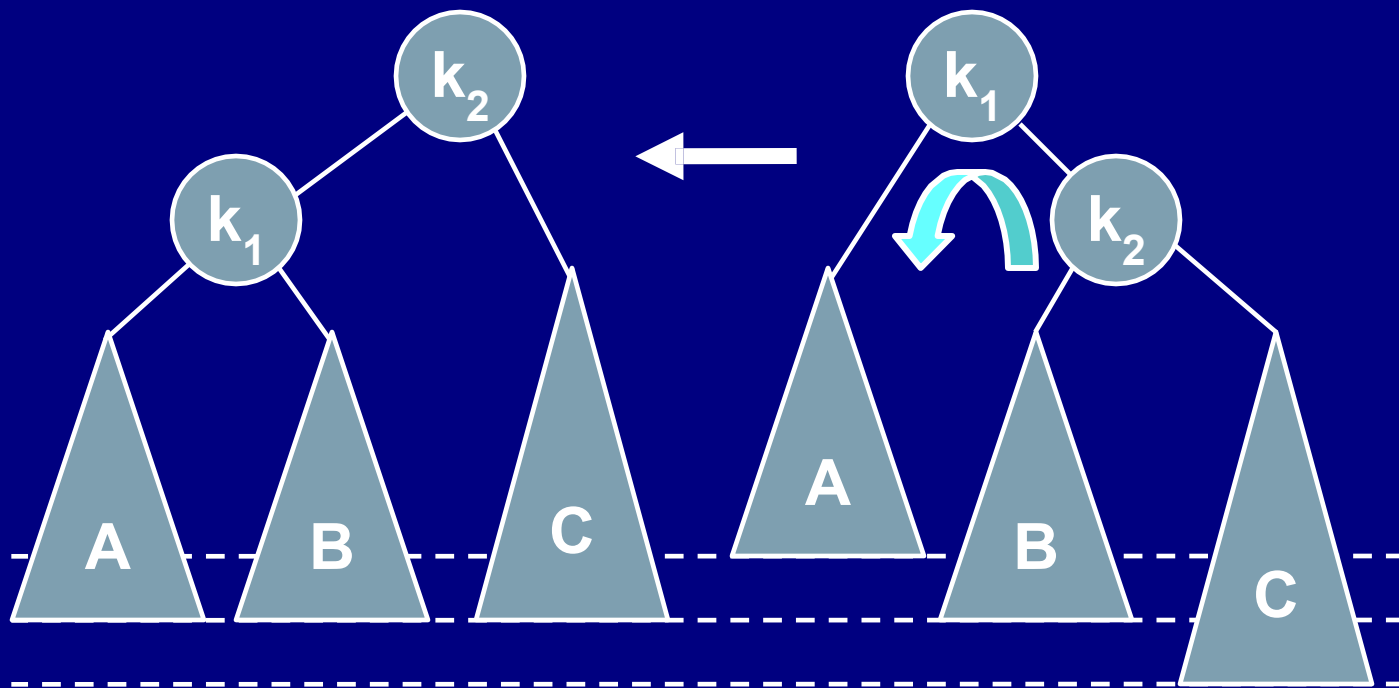
Single Rotation (case 1)

$$H_A = H_B + 1$$
$$H_B = H_C$$



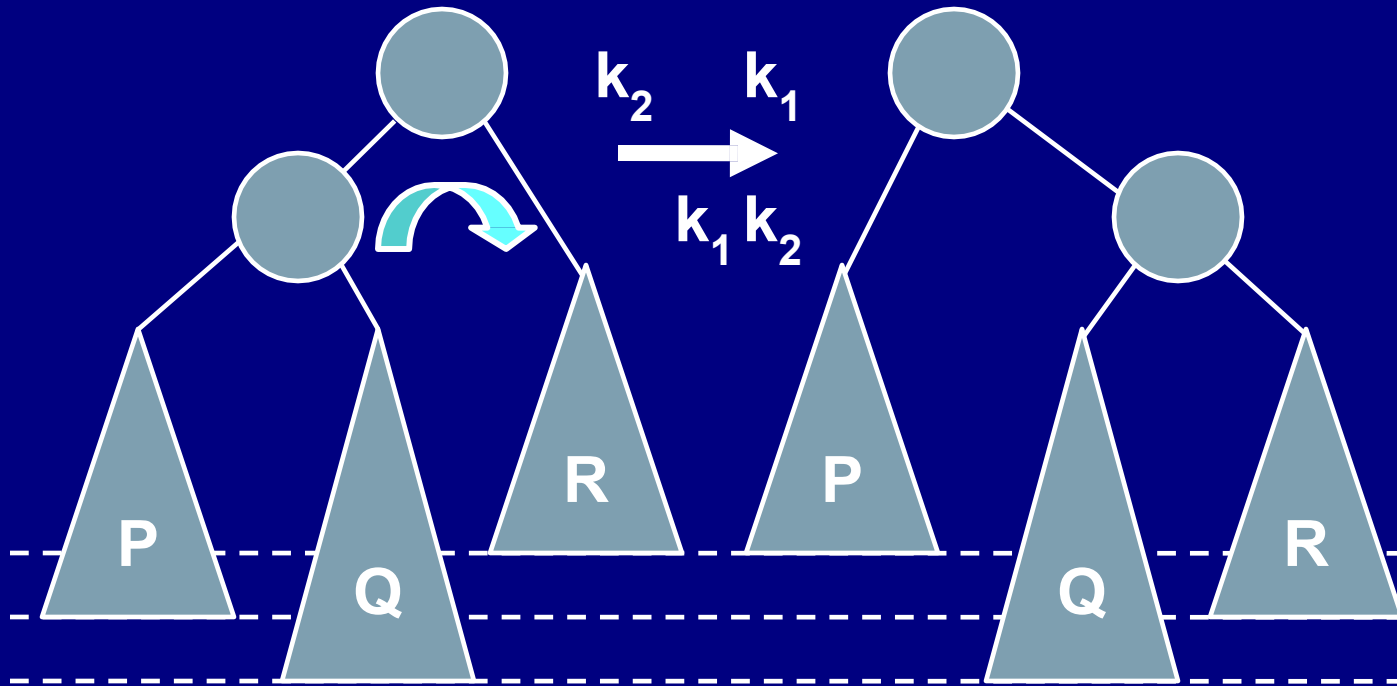
Single Rotation (case 4)

$$H_A = H_B$$
$$H_C = H_B + 1$$



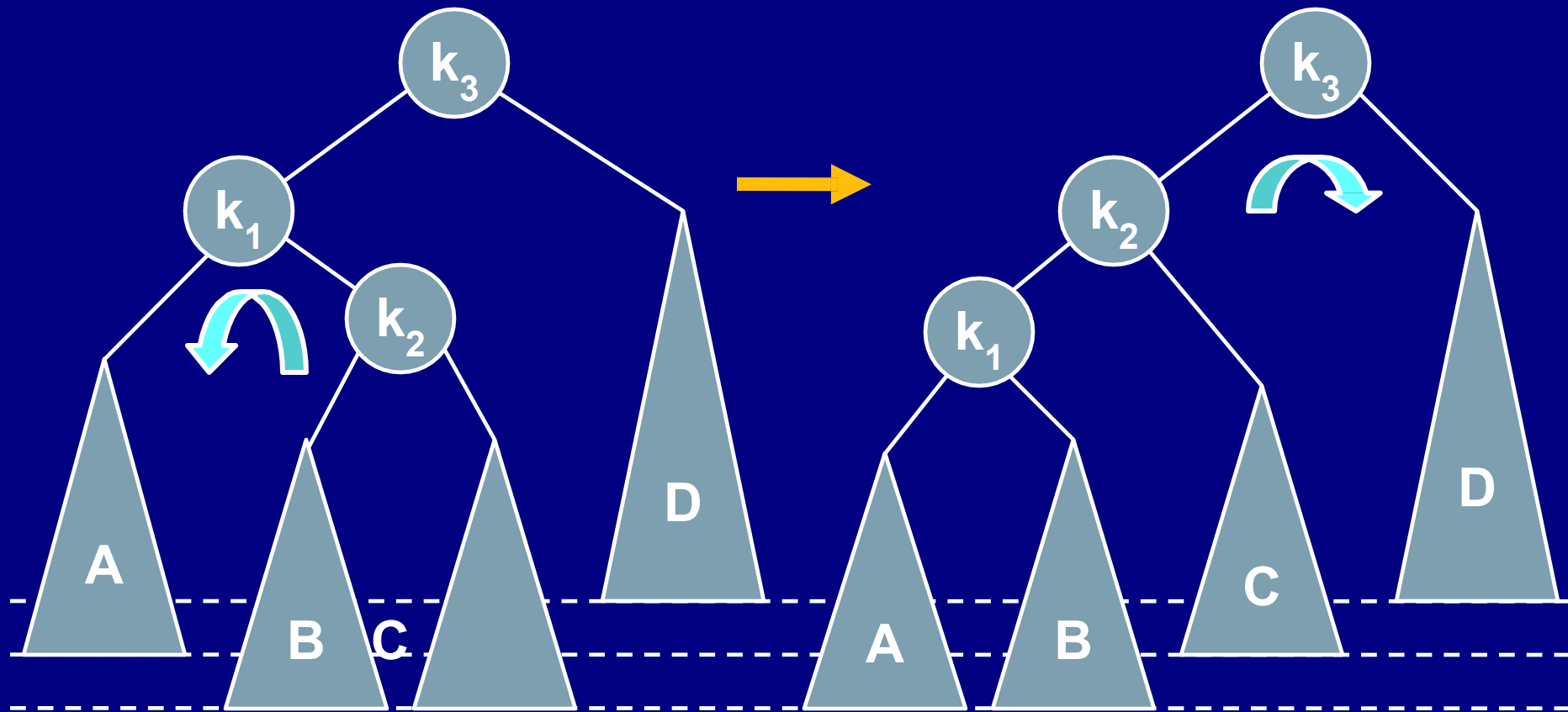
Keterbatasan Single Rotation

- Single rotation tidak bisa digunakan untuk kasus 2 dan 3 (*inside case*)



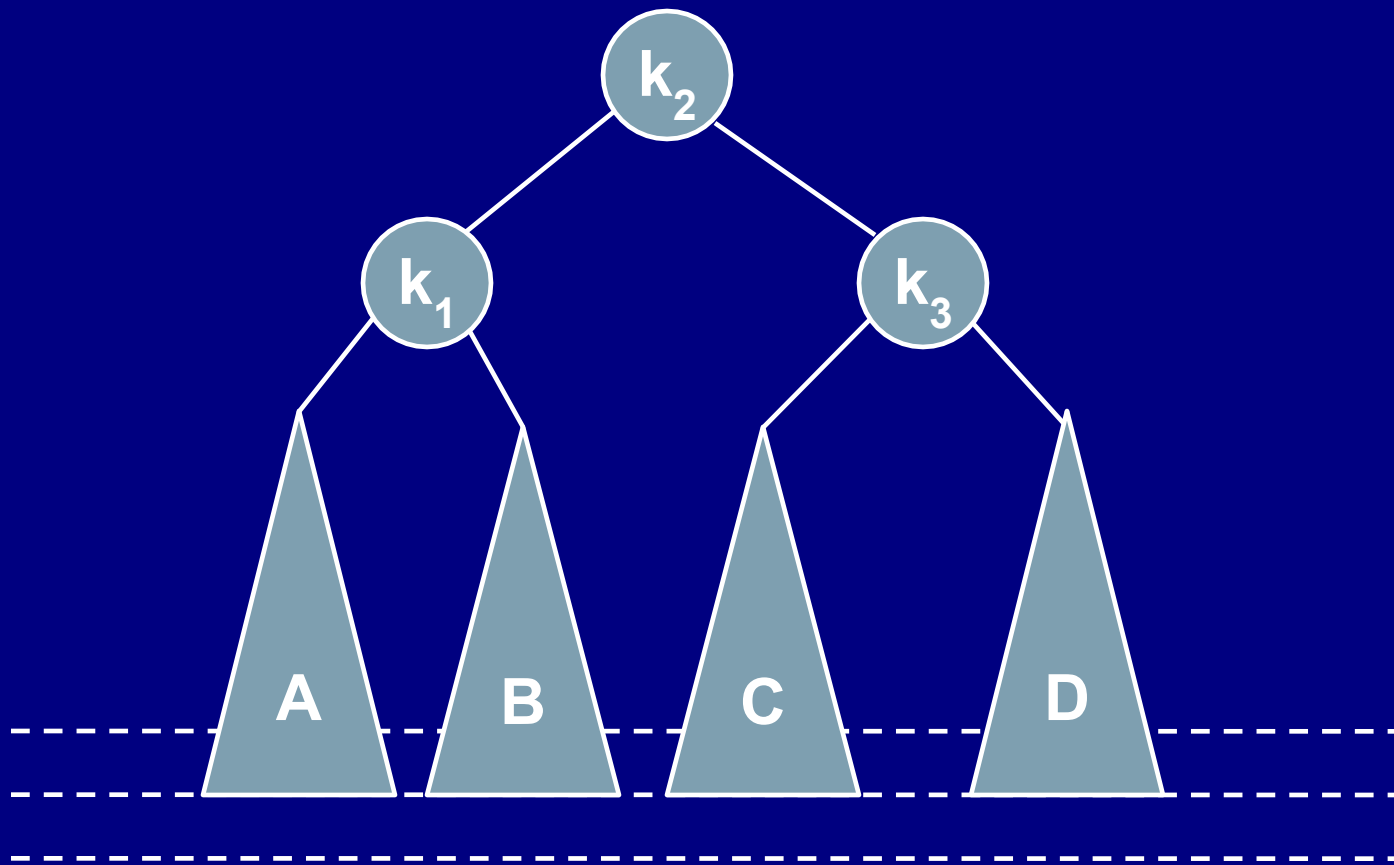
$$H_Q = H_P + 1$$
$$H_P = H_R$$

Double Rotation: Langkah

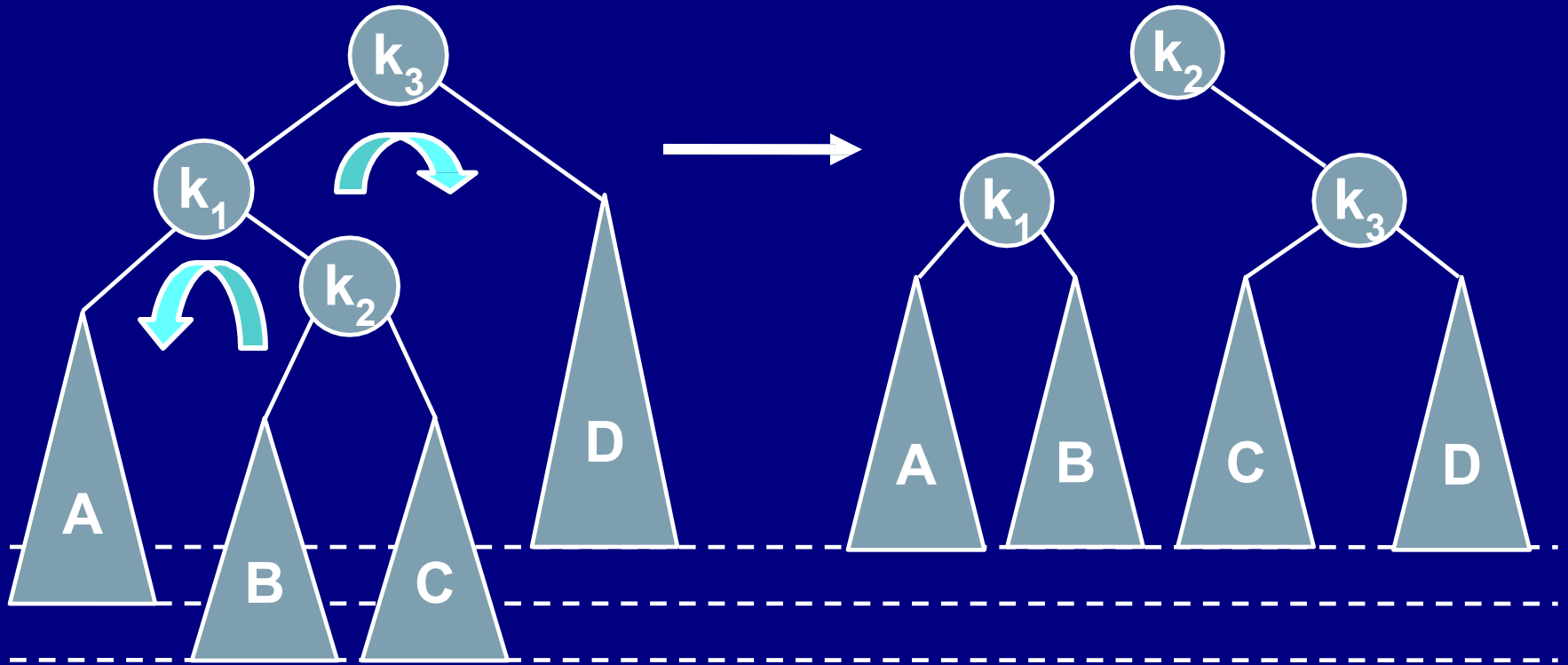


$$H_A = H_B = H_C = H_D$$

Double Rotation: Langkah

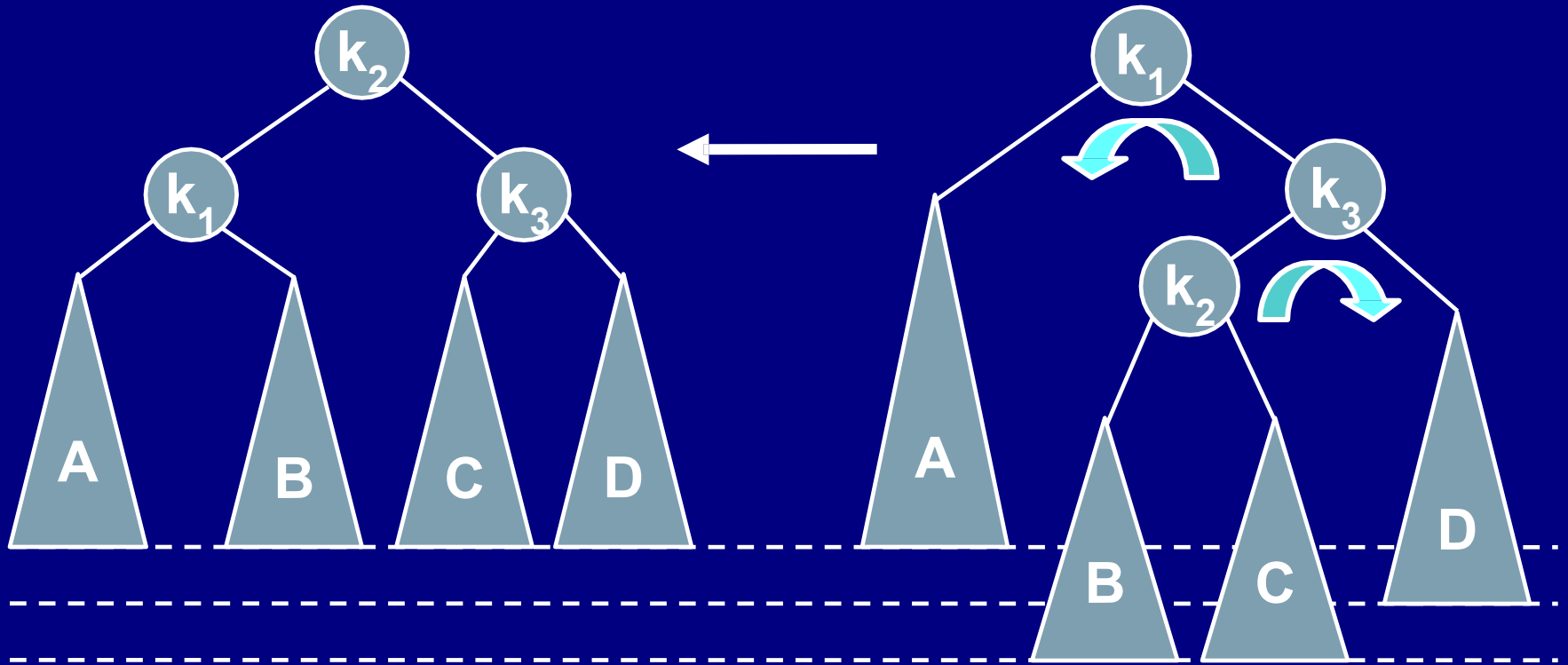


Double Rotation



$$H_A = H_B = H_C = H_D$$

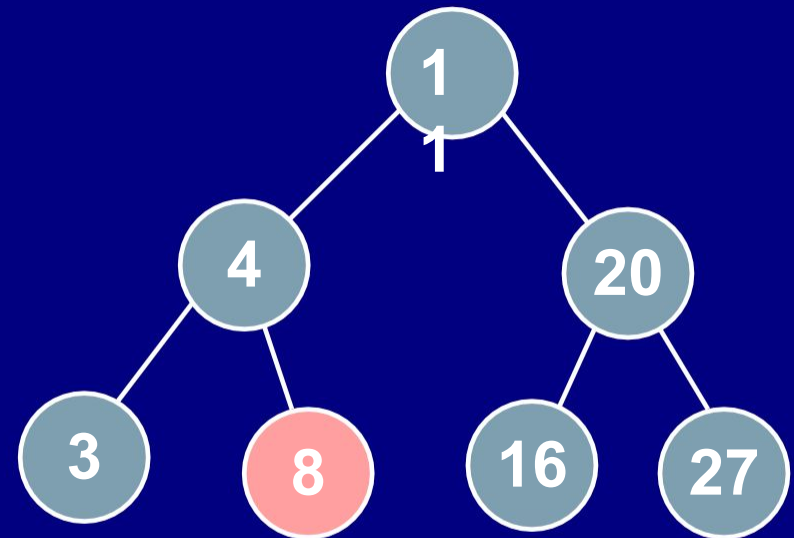
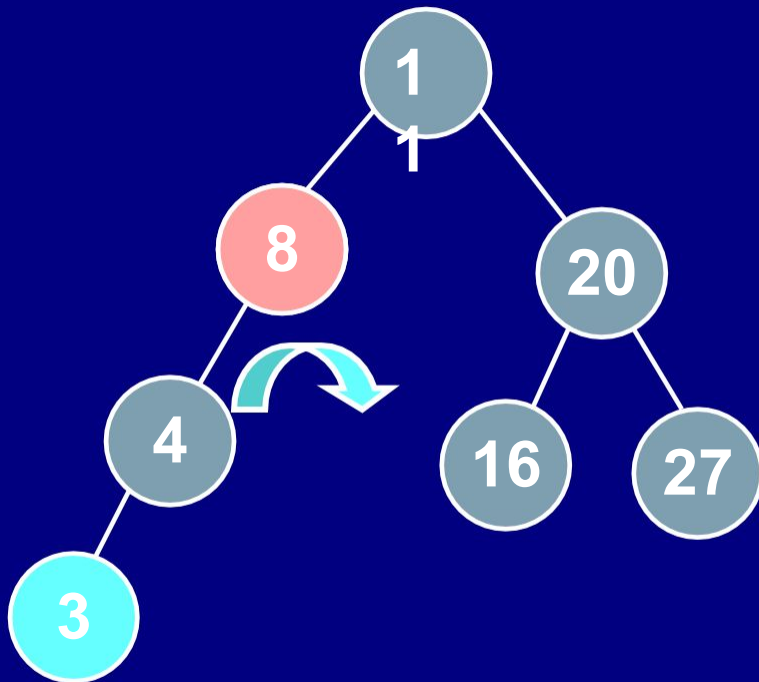
Double Rotation



$$H_A = H_B = H_C = H_D$$

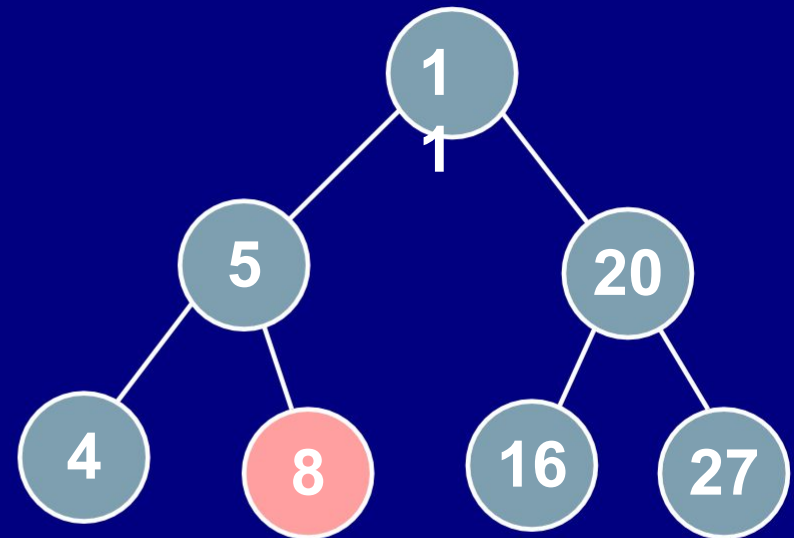
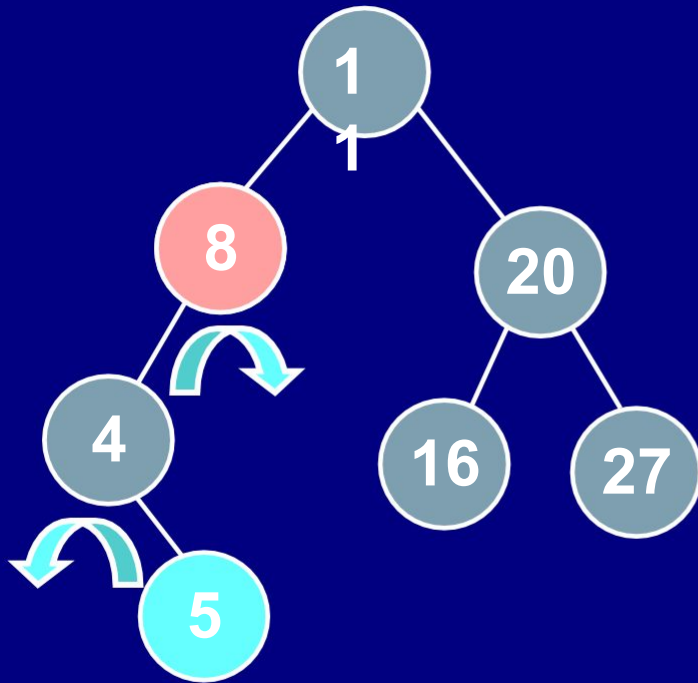
Contoh

- penambahan 3 pada AVL tree



Contoh

- penambahan 5 pada AVL tree



LATIHAN AVL-TREE (2)

- Coba simulasikan penambahan pada sebuah AVL dengan urutan penambahan:
 - 10, 85, 15, 70, 20, 60, 30, 50, 65, 80, 90, 40, 5, 55

LATIHAN AVL-TREE (1)

Coba simulasikan penambahan pada sebuah AVL dengan urutan penambahan:

10, 85, 15, 70, 20, 60, 30, 50, 65, 80, 90, 40, 5, 55

LATIHAN AVL-TREE (2)

Masukkan key berikut ke dalam AVL Tree :

51, 88, 92, 72, 41, 55, 35, 10